

P²C / PETC Core v1.1 — Production-Ready Witness Calculus for Tensor Provenance

This document consolidates the **P²C (Prime-Encoded Provenance Compiler)** core into a **minimal, verifiable, production-ready** specification and reference implementation.

It implements:

- **Signatures** as elements of a **free abelian group** over atoms.
- A **weak vs strong** notion of tensor-type equality.
- A **witness calculus** for reshape/contraction legality.
- **Verification-by-execution** (the witness interpreter is the single source of truth).
- **Cross-language stable commitments** and a **bytecode header** format.

1) Core idea

Each axis carries a **signature** (a sparse integer vector over atoms). Tensor programs are correct if legal reshapes and contractions are accompanied by a **witness** that can be verified cheaply.

Why two canonical forms?

- **Weak equivalence** is the reshape relation generated by `{Permute, Merge, Split}`.
- **Strong equality** preserves axis structure (useful for layout/sharding guarantees and optimization invariants).

Hierarchy:

Strong equality $\Rightarrow$ Weak equivalence $\Rightarrow$ No relation
($\approx$) ($\sim$)

2) Mathematical model

Signatures

Let `A` be a set of atoms.

- Signature group: $G = \mathbb{Z}^{(A)}$ (free abelian group; finitely supported maps `A` $\rightarrow$ $\mathbb{Z}$).
- Axis signature: $e \in G$.
- Tensor type: a finite list of axis signatures $T = [e_1, \dots, e_n]$.
- Global signature: $\Sigma(T) = \sum_i e_i$.

Reshape completeness theorem

Theorem (Reshape completeness). The equivalence relation on axis-lists generated by `{Permute, Merge, Split}` is exactly **global signature equality**:

$$T_1 \sim T_2 \text{ iff } \Sigma(T_1) = \Sigma(T_2).$$

Proof sketch.

1. Each primitive preserves Σ .
2. If $\Sigma(T_1) = \Sigma(T_2)$, merge all axes of each to a single axis $[\Sigma]$, then split into the target axes.

This is what justifies:

- `reshape_equivalent(src, tgt) := canonical_weak(src) == canonical_weak(tgt)`
- a complete synthesis strategy: `MergeAll ; SplitToTarget`

3) Prime encoding (optional codec)

The system's ground truth is the sparse-vector form $e \in \mathbb{Z}^{(A)}$. Prime products are an **optional codec**, not the canonical representation.

For duals, do **not** use rationals at runtime; encode as `(num, den)`:

- $M_+(e) = \prod_a p(a)^{\max(e(a), 0)}$
- $M_-(e) = \prod_a p(a)^{\max(-e(a), 0)}$

Equality check: `a.num * b.den == b.num * a.den`.

4) Production invariants

Scalar representation: `[]` (empty axis list).

Enforced everywhere:

- No empty axes
- No zero exponents
- All signatures are pruned

This guarantees stable equality, stable hashing, and predictable runtime behavior.

5) Reference implementation (Python)

```
"""
P2C Core v1.1 – Production Ready
=====
Signatures:  $G = \mathbb{Z}^A$  (free abelian group on atoms A)
Tensor type T:  $[e_1, \dots, e_n]$  where  $e_i \in G$ , all  $e_i$  non-empty and pruned
Scalar representation: [] (empty list)
"""

from __future__ import annotations
from dataclasses import dataclass
from collections import defaultdict
from typing import Dict, List, Tuple, Optional
import hashlib

# ===== Core Types =====

Atom = int
Sig = Dict[Atom, int]
Axes = List[Sig]

# ===== Invariant Enforcement =====

def sig_prune(s: Sig) -> Sig:
    return {k: v for k, v in s.items() if v != 0}

def sanitize_axes(axes: Axes) -> Axes:
    out: Axes = []
    for ax in axes:
        pruned = sig_prune(ax)
        if pruned:
            out.append(pruned)
    return out # scalar = []

# ===== Signature Operations =====

def sig_add(a: Sig, b: Sig) -> Sig:
    out = defaultdict(int)
    for k, v in a.items():
        out[k] += v
    for k, v in b.items():
        out[k] += v
    return sig_prune(dict(out))

def sig_neg(a: Sig) -> Sig:
```

```

        return sig_prune({k: -v for k, v in a.items()})

def sig_equal(a: Sig, b: Sig) -> bool:
    return sig_prune(a) == sig_prune(b)

def global_sig(axes: Axes) -> Sig:
    out = defaultdict(int)
    for ax in axes:
        for k, v in ax.items():
            out[k] += v
    return sig_prune(dict(out))

# ===== Canonical Forms =====

def canonical_weak(axes: Axes) -> Axes:
    g = global_sig(axes)
    return [{a: e} for a, e in sorted(g.items()) if e != 0]

def axis_key(ax: Sig) -> tuple:
    ax = sig_prune(ax)
    return tuple((a, ax[a]) for a in sorted(ax.keys()))

def canonical_strong(axes: Axes) -> Axes:
    normalized = [sig_prune(ax) for ax in axes]
    normalized = [ax for ax in normalized if ax]
    return sorted(normalized, key=axis_key)

def reshape_equivalent(src: Axes, tgt: Axes) -> bool:
    return canonical_weak(src) == canonical_weak(tgt)

# ===== Witness Language =====

@dataclass(frozen=True)
class Witness:
    pass

@dataclass(frozen=True)
class Permute(Witness):
    perm: Tuple[int, ...]

@dataclass(frozen=True)
class Merge(Witness):
    idxs: Tuple[int, ...]

@dataclass(frozen=True)
class Split(Witness):
    idx: int
    parts: Tuple[Sig, ...]

```

```

@dataclass(frozen=True)
class Contract(Witness):
    left: int
    right: int

@dataclass(frozen=True)
class Seq(Witness):
    steps: Tuple[Witness, ...]

# ===== Witness Application =====

def apply_witness(axes: Axes, w: Witness) -> Axes:
    axes = sanitize_axes([ax.copy() for ax in axes])

    if isinstance(w, Seq):
        for step in w.steps:
            axes = apply_witness(axes, step)
        return sanitize_axes(axes)

    if isinstance(w, Permute):
        if sorted(w.perm) != list(range(len(axes))):
            raise ValueError("Permute must be a permutation")
        axes = [axes[i] for i in w.perm]
        return sanitize_axes(axes)

    if isinstance(w, Merge):
        idxs = list(w.idxs)
        if len(idxs) < 2:
            raise ValueError("Merge requires at least two indices")
        if len(set(idxs)) != len(idxs):
            raise ValueError("Merge indices must be unique")
        if any(i < 0 or i >= len(axes) for i in idxs):
            raise ValueError("Merge index out of range")

        merged: Sig = {}
        for i in idxs:
            merged = sig_add(merged, axes[i])

        for i in reversed(sorted(idxs)):
            del axes[i]
        axes.insert(min(idxs), merged)
        return sanitize_axes(axes)

    if isinstance(w, Split):
        if w.idx < 0 or w.idx >= len(axes):
            raise ValueError("Split index out of range")
        original = axes[w.idx]

```

```

    parts = [sig_prune(p) for p in w.parts]
    if not parts or not all(parts):
        raise ValueError("Split parts must be non-empty")

    total: Sig = {}
    for p in parts:
        total = sig_add(total, p)
    if not sig_equal(total, original):
        raise ValueError("Split parts don't sum to original")

    del axes[w.idx]
    for j, p in enumerate(parts):
        axes.insert(w.idx + j, p)
    return sanitize_axes(axes)

if isinstance(w, Contract):
    if (w.left < 0 or w.right < 0 or
        w.left >= len(axes) or w.right >= len(axes) or
        w.left == w.right):
        raise ValueError("Contract indices out of range or equal")

    left_sig = axes[w.left]
    right_sig = axes[w.right]
    if not sig_equal(right_sig, sig_neg(left_sig)):
        raise ValueError("Contract requires dual axes")

    i, j = sorted((w.left, w.right))
    del axes[j]
    del axes[i]
    return sanitize_axes(axes)

raise TypeError(f"Unknown witness type: {type(w)}")

# ===== Reshape Synthesis (Exact-to-Target) =====

def permutation_to_match(src: Axes, tgt: Axes) -> Optional[Tuple[int, ...]]:
    buckets: Dict[tuple, List[int]] = {}
    for i, ax in enumerate(src):
        buckets.setdefault(axis_key(ax), []).append(i)

    perm: List[int] = []
    for ax in tgt:
        k = axis_key(ax)
        lst = buckets.get(k)
        if not lst:
            return None
        perm.append(lst.pop(0))

```

```

    return tuple(perm)

def synthesize_reshape_exact(src: Axes, tgt: Axes) -> Optional[Witness]:
    src = sanitize_axes(src)
    tgt = sanitize_axes(tgt)

    if canonical_weak(src) != canonical_weak(tgt):
        return None

    # Strong-equal => Permute or identity
    if canonical_strong(src) == canonical_strong(tgt):
        if src == tgt:
            return Seq(())
        perm = permutation_to_match(src, tgt)
        if perm is None:
            return None
        return Permute(perm) if perm != tuple(range(len(src))) else Seq(())

    steps: List[Witness] = []

    # MergeAll if needed
    if len(src) > 1:
        steps.append(Merge(tuple(range(len(src)))))

    # Map to target
    if len(tgt) == 0:
        # Scalar target; MergeAll may cancel to scalar
        return Seq(tuple(steps)) if steps else Seq(())

    if len(tgt) == 1:
        # Single-axis target; MergeAll yields it (under weak equivalence)
        return Seq(tuple(steps)) if steps else Seq(())

    steps.append(Split(0, tuple(tgt)))
    return Seq(tuple(steps))

# ===== Commitments (Cross-language stable encoding)
# =====

WEAK_MAGIC    = b'P2C_SIG_V1_WEAK\0'    # 16 bytes
STRONG_MAGIC  = b'P2C_SIG_V1_STRONG'    # 16 bytes
assert len(WEAK_MAGIC) == 16
assert len(STRONG_MAGIC) == 16

def encode_varint(value: int) -> bytes:
    if value < 0:
        raise ValueError("encode_varint expects non-negative integers")
    buf = bytearray()

```

```

while value >= 0x80:
    buf.append((value & 0x7F) | 0x80)
    value >>= 7
buf.append(value)
return bytes(buf)

def encode_signed(value: int) -> bytes:
    # ZigZag
    value = ((-value << 1) | 1) if value < 0 else (value << 1)
    return encode_varint(value)

def encode_weak_commitment(axes: Axes) -> bytes:
    axes = sanitize_axes(axes)
    g = global_sig(axes)
    pairs = [(a, g[a]) for a in sorted(g.keys()) if g[a] != 0]

    buf = bytearray()
    buf.extend(WEAK_MAGIC)
    buf.extend(encode_varint(len(pairs)))
    for atom, exp in pairs:
        buf.extend(encode_varint(atom))
        buf.extend(encode_signed(exp))
    return bytes(buf)

def encode_strong_commitment(axes: Axes) -> bytes:
    axes = sanitize_axes(axes)
    canonical = canonical_strong(axes)

    buf = bytearray()
    buf.extend(STRONG_MAGIC)
    buf.extend(encode_varint(len(canonical)))
    for ax in canonical:
        atoms = sorted(ax.keys())
        buf.extend(encode_varint(len(atoms)))
        for atom in atoms:
            buf.extend(encode_varint(atom))
            buf.extend(encode_signed(ax[atom]))
    return bytes(buf)

def commitment_digest(data: bytes) -> bytes:
    # BLAKE2b supports personalization; must be <= 16 bytes.
    return hashlib.blake2b(data, digest_size=32, person=b'P2C_2024').digest()

# ===== Witness Bytecode Header (v1.1) =====

WITNESS_MAGIC = b'P2CWITv1' # 8 bytes

def encode_witness_header(version: int, flags: int, body_len: int) -> bytes:

```



```

if body_len < 0 or body_len >= 2**32:
    raise ValueError("body_len must fit in uint32")
return (
    WITNESS_MAGIC +
    version.to_bytes(2, 'little') +
    flags.to_bytes(2, 'little') +
    body_len.to_bytes(4, 'little')
)

```

Note: Exactness is **modulo sanitization**, since the interpreter enforces invariants (empty/zero components are normalized away).

6) Runtime embedding: verifier design notes

A runtime verifier should:

- Parse the 16-byte header (magic, version, flags, body length).
- Decode opcodes in a single pass.
- Reject malformed bytecode via bounds checks.
- Enforce merge uniqueness, permute correctness, and index range checks.

Implementation strategy:

- Keep axis signatures as small hashmaps or sorted vectors.
- Use a scratch buffer to accumulate merged sums, then prune zeros.

7) Extension points (PETC layer)

Coarsening as pushforward

A coarsening policy is a map $q : \text{Atom} \rightarrow \text{Atom}'$ inducing a homomorphism:

$$q_* e(b) = \sum_{a: q(a)=b} e(a)$$

Govern coarsenings via an **audited DAG** and require explicit module imports.

Effects

Keep a separate effect system for basis-mixing transforms (FFT, learned projections, etc.) so signature correctness does not imply semantic safety.

Distributed commitments

- Store weak commitments per shard.

- Maintain a global commitment for consistency checks across reshard/allgather/allreduce.

8) External references

Below are relevant background references for the major ingredients (proof-carrying artifacts, hashing commitments, varint encodings, and distributed tensor IR/sharding).

Citations (human-readable)

- George C. Necula (1997): *Proof-Carrying Code*. POPL.
- RFC 7693 (2015): *The BLAKE2 Cryptographic Hash and Message Authentication Code (MAC)*.
- Protocol Buffers documentation: *Encoding* (Varints + ZigZag).
- OpenXLA: *StableHLO Specification*.
- Xu et al. (2021): *GSPMD: General and Scalable Parallelization for ML Computation Graphs*.
- JAX documentation: `jax.sharding` (Mesh + PartitionSpec/NamedSharding).
- PyTorch/XLA blog: *PyTorch/XLA SPMD*.
- Bernardy et al. (2017): *Linear Haskell: practical linearity in a higher-order polymorphic language*.
- Brady: *Idris, a General Purpose Dependently Typed Programming Language*.

URLs (kept in a code block for portability)

```
Proof-Carrying Code (ACM): https://dl.acm.org/doi/10.1145/263699.263712
Proof-Carrying Code (PDF mirror): https://courses.grainger.illinois.edu/cs421/
fa2010/papers/necula-pcc.pdf
RFC 7693 (IETF): https://datatracker.ietf.org/doc/html/rfc7693.html
Protocol Buffers Encoding (Varint/ZigZag): https://protobuf.dev/programming-
guides/encoding/
StableHLO Spec (OpenXLA): https://openxla.org/stablehlo/spec
StableHLO GitHub: https://github.com/openxla/stablehlo
GSPMD paper (arXiv): https://arxiv.org/abs/2105.04663
JAX sharding docs: https://docs.jax.dev/en/latest/jax.sharding.html
PyTorch/XLA SPMD blog: https://pytorch.org/blog/pytorch-xla-spmd/
Linear Haskell (ACM): https://dl.acm.org/doi/10.1145/3158093
Linear Haskell (arXiv): https://arxiv.org/abs/1710.09756
Idris implementation paper (PDF): https://www.type-driven.org.uk/edwinb/papers/
impldtp.pdf
```